University Housing Administration System

System Analysis and Design Document (Enterprise Level)

Project Overview

✨ Project Vision

The University Housing Administration System is an enterprise-grade web application aimed at automating and managing student housing operations, including applications, algorithmic room allocation, targeted payments, and maintenance lifecycle management.

Sponsored by the Government (Ministry of Higher Education), this project heavily supports digital transformation and smart campus initiatives.

Key Architectural Pillars:

- **Dynamic Scalability:** Architected to easily scale and be adopted by other universities nationwide.
- **Security & Compliance:** Built with enterprise-level security protocols to ensure strict protection of sensitive student data (in compliance with national privacy standards).

Project Structure Overview

```
📂 MY PROJECT
    📂 Planning
        📄 System Request   (Chapter 1: Project Initiation)
        📄 Feasibility Analysis
        📄 Project Workplan   (Chapter 2: Project Management)
        📄 Staffing Plan
        📄 Project Standards
        📄 Risk Assessment
    📂 Analysis
    📂 Design
    📂 Implementation
```

System Development Life Cycle (SDLC)

💡 Idea → Planning → Analysis → Design → Implementation → ✅ Success

Chapter 1: Planning

1. Technical Feasibility

The system leverages a hybrid approach: **Node.js** for robust backend operations and **Firebase** for rapid Cloud DB/Auth. This ensures scalable server-side data processing combined with secure, fast cloud infrastructure.

2. Economic Feasibility

Currently, the university incurs massive overhead processing paperwork. Transitioning to a serverless/cloud-managed infrastructure (Firebase) drastically reduces physical server costs and eliminates thousands of hours of administrative overtime pay.

3. Organizational Feasibility

Students heavily favor digital platforms over traveling hours to Cairo simply to submit housing papers. The alignment with the Ministry's digital transformation roadmap guarantees high adoption rates from both the administrative body and the student population.

Next Steps in SDLC

Following the Planning phase, the project strictly progresses through:

- **Analysis:** Defining rigorous quantitative problems and exact functional requirements.
- **Design:** Formulating backend algorithms (Room Allocation) and cloud database schemas.
- **Implementation:** Writing custom business logic in Node.js and binding it to Firebase.
- **Testing:** Verifying security compliance and end-to-end payment simulations.

Chapter 2: System Modeling

2.1 System Actors

- **Student:** A verified university student applying for housing.
- **Admin (Housing Administrator):** Reviews applications, overrides algorithmic allocations if necessary.
- **Payment Gateway (External System):** Handles transaction processing and responds with success/failure tokens (Simulation mode during development).
- **Government (Sponsor):** Indirect actor providing funding metrics.

2.2 Student Use Cases & Preconditions

*Crucial System Logic: All functional actions (Status, Allocation, Payments) share a strict **precondition**: The user MUST be securely authenticated. Therefore, 'Login' is a precondition, not an optional 'extend' behavior.*

- Register Account / Login (Authentication phase)
- Upload Verified Documents
- Check Application Status
- View Room Allocation
- Execute Digital Payment via Gateway
- Submit Maintenance Request

System Modeling (Continued)

2.3 Admin Use Cases

- Dashboard Authentication
- Bulk Data Review (Applications & Docs)
- Accept / Reject Logic processing
- Trigger Algorithmic Room Assignment
- Override Room Assignment (Manual fail-safe)
- Review Global Payment Ledgers
- Dispatch Maintenance Crews

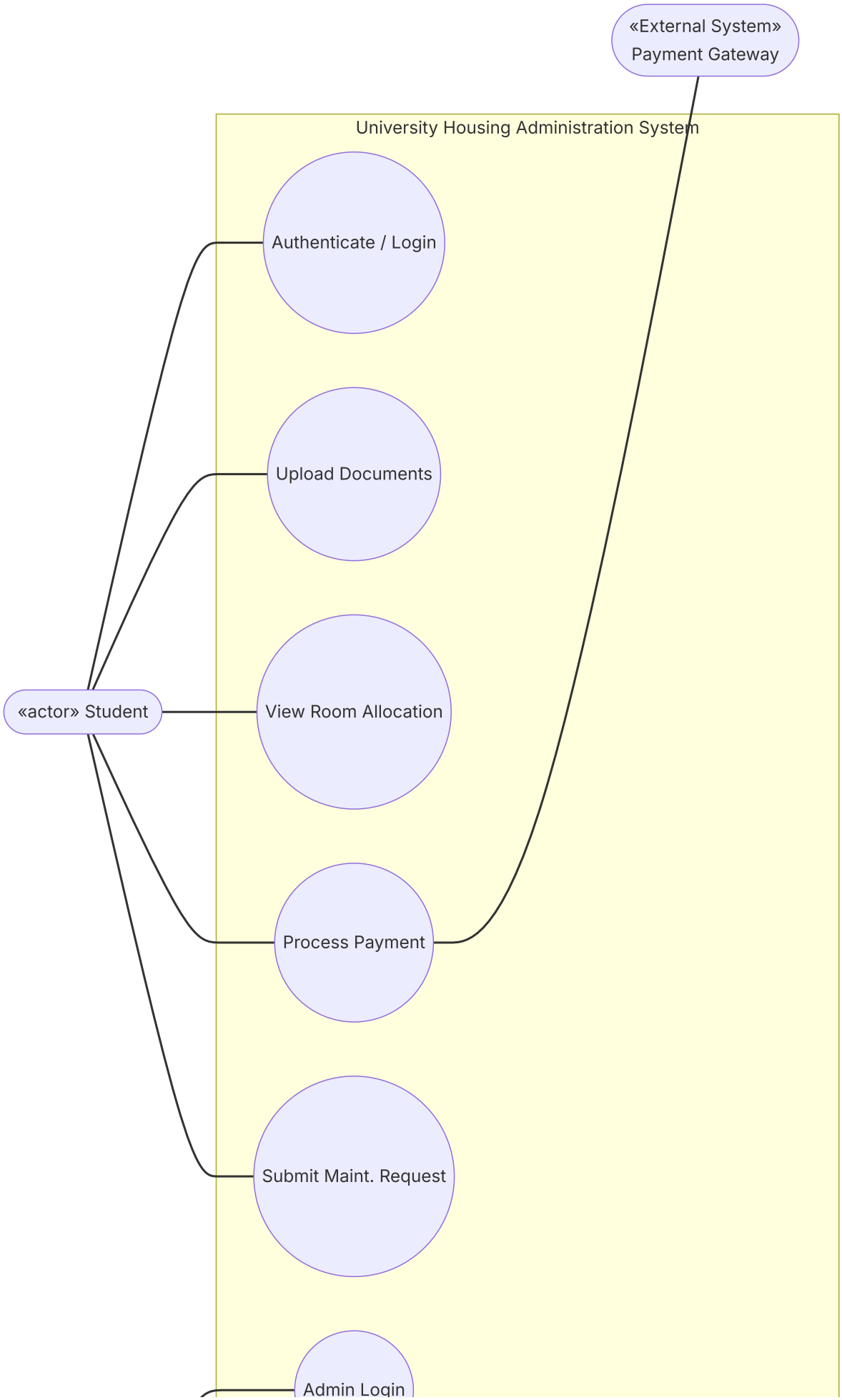
2.4 Entity Cardinality Rules (ERD Specifications)

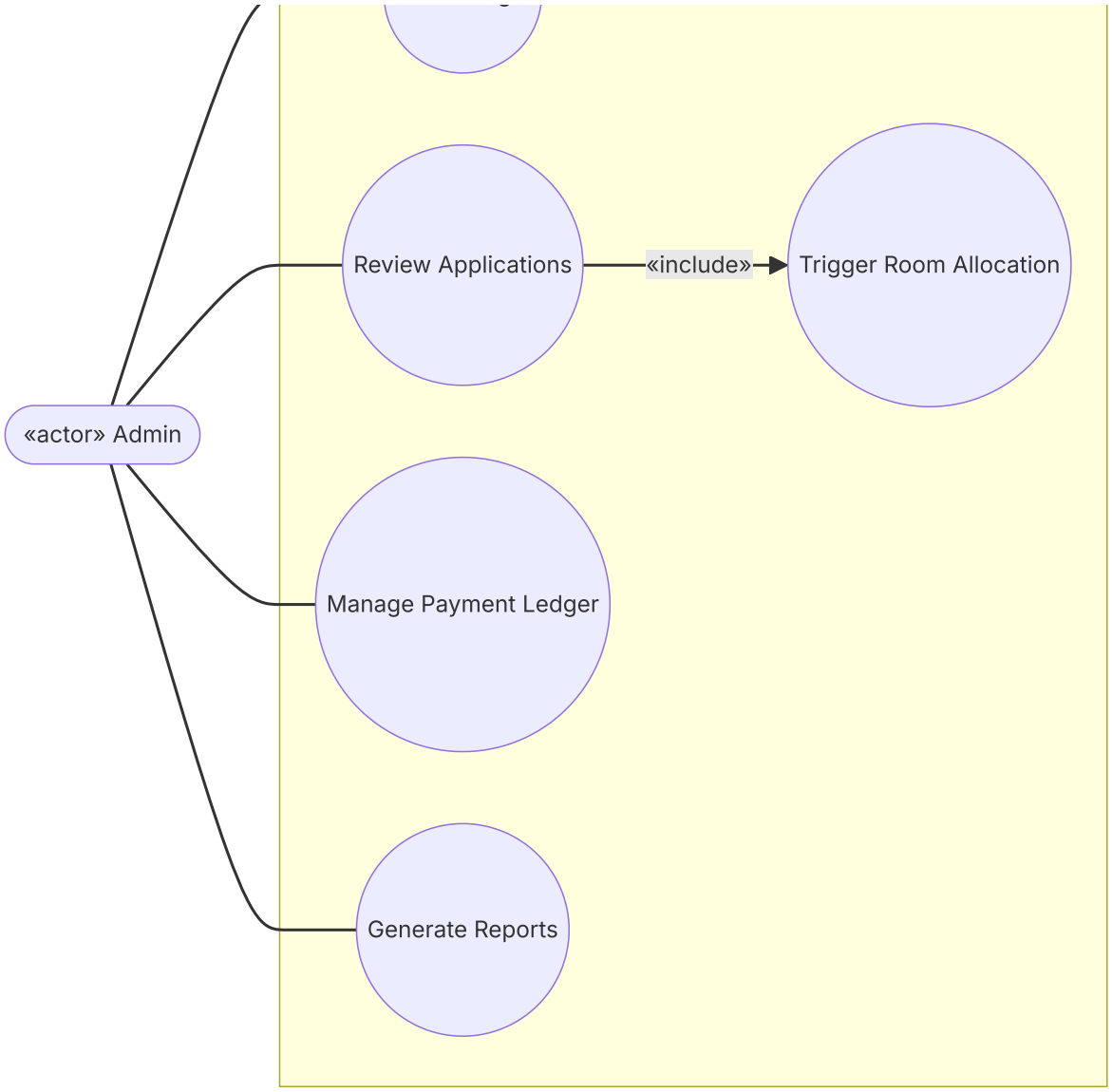
The database strictly follows these cardinality mapping rules to ensure data normalization:

- **1 STUDENT** to **Many (0..*) APPLICATIONS** (Students can reapply yearly).
- **1 APPLICATION** contains **1..* DOCUMENTS**.
- **1 ROOM** hosts **Many (0..MAX_CAPACITY) STUDENTS**. Status strictly toggles to "FULL" based on current_occupancy.
- **1 STUDENT** generates **Many (0..*) PAYMENTS** (Monthly/Yearly installements).
- **ROOM** classification adheres to strict `gender_restriction` checks against the Student profile during allocation.

2.5 Use Case Diagram

Enterprise Use Case Architecture

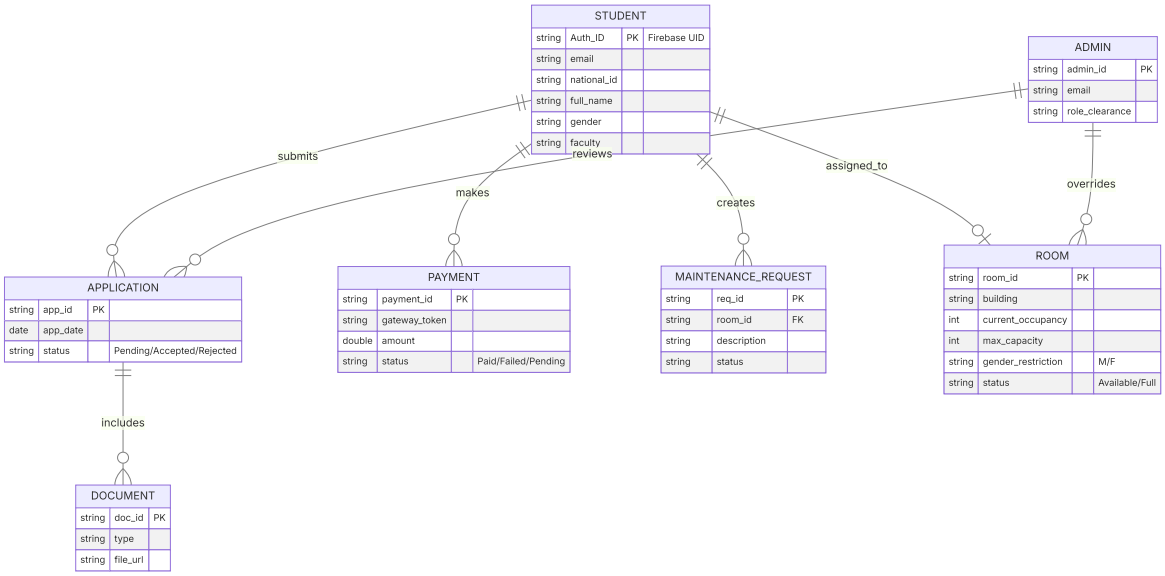

«External System»
Payment Gateway
University Housing Administration System
Authenticate / Login
Upload Documents
«actor» Student
View Room Allocation
Process Payment
Submit Maint. Request
Admin Login



Note: Authentication is treated as a core system precondition for protected routes, rather than an optional <<extend>> relationship. External Payment Gateway handles financial flows.

2.6 Entity-Relationship (ER) Diagram

Logical Database Schema



Comprehensive mappings include strict tracking of current_occupancy vs max_capacity, and Auth_ID mapping directly to Firebase authentication blocks.

Chapter 3: Analysis Phase

3.1 Problem Statement (Quantitative Analysis)

The current manual housing system faces severe administrative bottlenecks:

- **Over 1,000+ applications are submitted manually every semester.**
- **Manual paperwork processing and verification takes an average of 2–3 weeks.**
- **Room allocation errors (double-booking) occur frequently due to unlinked ledger systems.**

3.2 Advanced Technology Stack Justification

The solution utilizes a highly optimized hybrid stack:

Backend Logic: Node.js | Database & Auth: Firebase | Frontend: React / Web

System Architecture Question: "Why not pure Firebase?"

While Firebase provides rapid Firestore data storage and excellent OAuth handling, **Node.js is introduced explicitly to handle custom, heavy business logic.**

For example, computing the **Algorithmic Room Allocation** (matching students based on faculty distance, gender restrictions, and dynamic room occupancy) is too complex and insecure for client-side Firebase rules. Node.js acts as a secure, scalable middleware to process these heavy calculations, prepare secure simulated payment requests, and interact with Firebase via the Admin SDK.

Analysis Phase (Continued)

3.3 Algorithmic Room Allocation & Payment Flow

🧠 Room Allocation Core Logic

Allocation is not a manual click-by-click process. When an Admin triggers allocation, the Node.js backend processes an algorithm that:

1. Filters available rooms by matching the student's `gender` to the room's `gender_restriction`.
2. Checks that `current_occupancy < max_capacity`.
3. Assigns the student, increments occupancy, and auto-toggles room status to "Full" if capacity is met.

💳 Payment Simulation

In the analysis phase, real credit card processing is substituted with a simulated API. The system generates a mocked Webhook response from an External Payment Gateway, updating the Firebase `PAYMENT` status from 'Pending' to 'Paid' and issuing a digital receipt.

3.4 Enterprise Business Value

- **Efficiency:** Cuts application processing time from 3 weeks to under 48 hours.
- **Data Integrity:** Eliminates 100% of double-booking errors via atomic database transactions.
- **Scalable Architecture:** Built to easily adopt 10,000+ simultaneous connections during peak enrollment days.

Chapter 4: Design Phase

4.1 Architecture Design (Data Flow)

Presentation Layer (Client)

User logs in. Firebase Auth returns a secure JWT token. Client requests data.

Business Logic Component (Node.js REST API)

Node.js receives the JWT, verifies it via Firebase Admin SDK, enforces Role-Based Access Control (Student vs Admin), and executes custom allocation algorithms or payment logic.

Data Layer (Firestore)

Node.js directly communicates securely with Firestore to update Student Ledgers and Room Occupancy arrays, abstracting db complexity from the client.

4.2 Interface & Experience Design

🎓 Student Portal

- Live application status tracker (Timeline view).
- Digital Payment Invoice view.
- Dynamic form validation (preventing submission of empty National IDs).

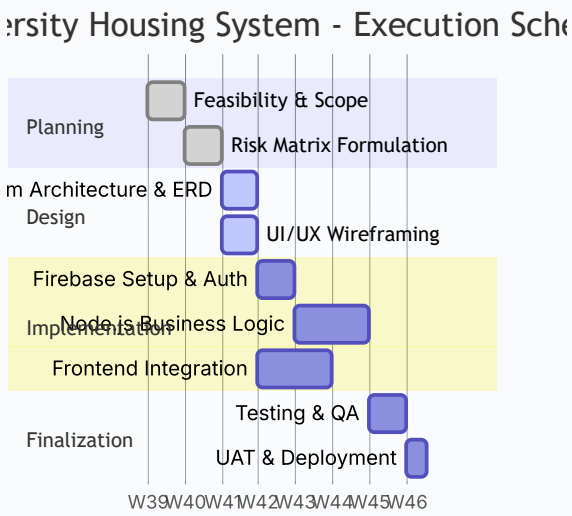
👨‍💼 Admin Command Center

- Data Tables with sorting/filtering by 'Pending' vs 'Verified'.
- One-click Algorithmic Assignment trigger.
- Occupancy Heatmap (Visual report of full buildings).

Chapter 5: Project Management

5.1 Project Gantt Chart (8 Weeks)

Visualizing the overlapping phases and scheduling of the SDLC.



Project Management (Continued)

5.2 Milestones & Critical Path Analysis

🎯 **Major Project Milestones**

1. **Week 2:** Requirements Signed-Off & Architecture Finalized.
2. **Week 4:** Core Firebase DB operations and Auth functional.
3. **Week 6:** Node.js Room Allocation Algorithm successfully tested.
4. **Week 8:** System Go-Live and Handover to University Admins.

🏔️ **The Critical Path**

The sequence of stages determining the minimum time required for operation is: *Database Design → Backend Auth → Node.js Allocation Logic → Frontend Binding → Security Testing.*
Any delay in the Node.js Allocation Logic will directly delay the final release, hence maximum developer resources will be dedicated to this segment.

5.3 Staffing & Standards

- **Project Manager:** Oversees timeline, mitigates critical path delays.
- **Node.js Backend Dev:** Implements custom constraints and algorithmic routing.
- **Firebase Archtiect:** Sets up Firestore security rules and collections.
- **Standards:** Strict adherence to RESTful API conventions, Git branching strategy (Dev/Staging/Main), and OWASP security guidelines.

Chapter 6: Risk Management

6.1 Comprehensive Risk Matrix

Analyzing potential technical and organizational failures before they occur.

Risk Event	Probability	Impact (1-5)	Risk Level	Mitigation Strategy
Node.js Algorithm Failure (Incorrect room assignments)	30%	5 (Severe)	HIGH	Implement strict Test-Driven Development (TDD) and mock data simulations before production merge.
Data Breach / Unauthorized Access	15%	5 (Severe)	HIGH	Enforce Firebase Security Rules, JWT token verification, and encrypt environment variables.
Payment Gateway API Downtime	20%	3 (Moderate)	MEDIUM	Build a queueing system so students can 'pay later' without generating a system-wide crash.
Student Resistance to Digital System	40%	2 (Minor)	LOW	Provide intuitive UI/UX and launch comprehensive "How-To" video tutorials upon release.

*Risk Score = Probability x Impact. High risks are prioritized in weekly sprints.

Risk Management (Schedule)

6.2 Risk Mitigation Task Schedule

Risk handling tasks embedded directly into the 8-week SDLC operational schedule.

ID	Task Name	Duration	Start	Finish	Responsible Role
1	Formulate precise Risk Matrix parameters	3 days	Week 1	Week 1	Project Manager
2	Technical infrastructure stress testing	2 days	Week 1	Week 1	Backend Dev, Firebase Architect